Raport Końcowy

Przedmiot: Języki asemblerowe sem.5

Autor: Mateusz Smuda

Grupa: 5

Sekcja: 9

1. Informacje z karty projektu:

Temat projektu: Symulator Daltonizmu

Opis ogólny programu: Program symuluje daltonizm w obrazach z wykorzystaniem wielowątkowości. Dzięki programowi można zobaczyć jak osoby posiadające dany typ daltonizmu postrzegają barwy. Program zostanie zaimplementowany w języku C++ oraz w asemblerze dzięki czemu będzie możliwe porównanie efektywności dwóch implementacji programu. Program umożliwia symulacje deuteranopii(niedowidzenie koloru zielonego), protanopii(niedowidzenie koloru czerwonego), tritanopii(niedowidzenie koloru niebieskiego)

Cel projektu: Celem projektu jest stworzenie aplikacji, która pozwoli zobaczyć jak osoby chorujące na daltonizm postrzegają niektóre barwy. Dodatkowo chcemy zobaczyć czy pisanie programu w asemblerze przynosi korzyści pod względem szybkości działania w porównaniu do pisania w języku wysokiego poziomu takim jak C++. Będzie także możliwe zobaczenie jak liczba wątków wpływa na efektywność działania danego kodu.

Główne założenia projektu:

1) Wczytywanie obrazu: Użytkownik wgrywa plik graficzny w formacie jpg za pomocą interfejsu aplikacji. Obraz ten zostaje załadowany i przygotowany do przetwarzania.

2) Przetwarzanie obrazu:

Istnieją dwie opcje przetwarzania obrazu:

• Przetwarzanie w C++: Symulacja daltonizmu realizowana jest w języku C++, z wykorzystaniem wielowątkowości. Każdy wątek odpowiada za przetwarzanie jednej części obrazu, co pozwala na zwiększenie efektywności.

• Przetwarzanie w ASM: Symulacja daltonizmu została także zaimplementowana w asemblerze, co pozwala na szybsze przetwarzanie obrazu. Funkcja przetwarza obraz na poziomie bitów, co pozwala na optymalizację i przyspieszenie operacji.

3) Zapis przetworzonego obrazu: Po przetworzeniu obrazu użytkownik ma możliwość zapisania gotowego obrazu w formacie jpg.

Algorytmy symulowania poszczególnych rodzajów daltonizmu zgodne z macierzami modelu Machado 2009:

Algorytm symulacji deuteranopii opiera się na zmodyfikowaniu wartości poszczególnych kanałów kolorów:

• Kanał czerwony (R): R = 0.367 * R + 0.861 * G – 0.228 * B

• Kanał zielony (G): G = 0.280 * R + 0.673 * G + 0.047 * B

• Kanał niebieski (B): B = -0.012 * R + 0.043 * G + 0.969 * B Algorytm symulacji protanopii opiera się na zmodyfikowaniu wartości poszczególnych kanałów kolorów:

• Kanał czerwony (R): R = 0.152 * R + 1.052 * G – 0.205 * B • Kanał zielony (G): G = 0.115 * R + 0.786 * G – 0.099 * B

• Kanał niebieski (B): B = -0.003 * R – 0.048 * G + 1.052 * B Algorytm symulacji tritanopii opiera się na zmodyfikowaniu wartości poszczególnych kanałów kolorów: • Kanał czerwony (R): R = 1.256 * R - 0.077 * G – 0.179 * B

• Kanał zielony (G): G = -0.078 * R + 0.931 * G + 0.148 * B

• Kanał niebieski (B): B = 0.005 * R + 0.691 * G – 0.304 * B W ten sposób kolory są przekształcane, aby zbliżyć się do widzenia przez osobę z danym typem daltonizmu. Proces ten jest realizowany dla każdego piksela obrazu.

Interfejs graficzny: Aplikacja posiada graficzny interfejs użytkownika zaprojektowany z wykorzystaniem biblioteki Qt.

2. Opis problemu, oraz metody implementacji rozwiązania tego problemu:

Celem programu jest symulacja daltonizmu, czyli pokazanie zdjęcia tak, jak widzi je osoba z wadą wzroku. Współczesne zdjęcia składają się z wielu pikseli dlatego aby symulacja zadziałała, komputer musi dla każdego piksela wykonać odpowiednie działania matematyczne (przeliczyć kolory R, G, B według odpowiednich wzorów). Przy zwykłym podejściu (przetwarzanie piksel po pikselu w C++) zajmuje to dużo czasu, przez co program może działać wolno na dużych zdjęciach w przypadku gdy kompilator nie wybierze optymalnego rozwiązania. Głównym zadaniem projektu jest sprawdzenie, czy użycie Asemblera i instrukcji procesora, które wykonują działania na wielu danych jednocześnie, pozwoli przyspieszyć ten proces.

W celu maksymalizacji wydajności przetwarzania obrazu, implementacja niskopoziomowa (Asembler x64) została zaprojektowana w oparciu o architekturę hybrydową, łączącą przetwarzanie wektorowe (SIMD) z precyzyjnym przetwarzaniem skalarnym:

1) Wektoryzacja obliczeń (AVX2) – Pętla Główna

Główną częścią algorytmu jest pętla wykorzystująca rozszerzenie AVX2 (rejestry 256-bitowe YMM).

- Równoległość danych**:** W jednej iteracji pętli przetwarzane są jednocześnie 4 piksele (16 składowych koloru).
- Rozszerzanie danych: Ponieważ operacje arytmetyczne na liczbach 8-bitowych mogłyby doprowadzić do przepełnienia (overflow), dane wejściowe są rozszerzane „w locie” do formatu 16-bitowego (word) za pomocą instrukcji vpmovzxbw.

- Przetwarzanie macierzowe: Mnożenie wektora kolorów przez macierz przekształceń realizowane jest instrukcją vpmaddwd (Vector Multiply and Add), która wykonuje mnożenie i dodawanie w jednym cyklu zegarowym, oraz vphaddd do sumowania horyzontalnego.

2) Arytmetyka Stałoprzecinkowa (Fixed Point Q10)

W celu uniknięcia kosztownych operacji na liczbach zmiennoprzecinkowych (float), zastosowano format stałoprzecinkowy Q10.

- Współczynniki macierzy Machado zostały przemnożone przez 1024 i przechowywane jako liczby całkowite ze znakiem (signed word).
- Po wykonaniu mnożenia, wynik jest normalizowany poprzez przesunięcie bitowe w prawo o 10 pozycji (instrukcja vpsrad dla wektorów oraz sar dla skalarów), co jest operacją znacznie szybszą od dzielenia (div).

3) Obsługa krawędzi i Adresowanie Pamięci

Ze względu na to, że pętla wektorowa przetwarza bloki po 4 piksele, obrazy o szerokości niepodzielnej przez 4 wymagają specjalnej obsługi końcowych pikseli (tzw. reszty).

- Zaimplementowano Pętlę Skalarną (Scalar Loop), która przetwarza pozostałe 1-3 piksele.
- W pętli tej wykorzystano zaawansowany tryb adresowania procesora x64: [Base + Index * Scale]:

  W kodzie: movzx ebx, byte ptr [rsi + r9*4]. Pozwala to na swobodny dostęp do poszczególnych pikseli bez konieczności fizycznego przesuwania głównego wskaźnika RSI, co upraszcza logikę sterowania pętlą.

4) Saturacja (Clamping) i Obsługa błędów

Algorytm symulacji (szczególnie dla Tritanopii) może generować wartości ujemne lub przekraczające 255.

- Zaimplementowano dedykowaną procedurę Clamp8, która dokonuje saturacji wyników do zakresu bajta [0, 255].
- Zastosowano instrukcje warunkowe (js - jump if sign, jg - jump if greater) do szybkiej korekty wartości, co eliminuje błędy wizualne (tzw. artefakty przekręcenia licznika) na obrazie wynikowym.

5) Optymalizacja dostępu do danych

- Wyrównanie pamięci: Dane stałe (.data) są wyrównane do granicy 16 bajtów (ALIGN 16), co sprzyja szybszemu ładowaniu do rejestrów procesora.
- Tablica skoków (Lookup Table): Adresy macierzy współczynników przechowywane są w tablicy CoeffTable. Wybór odpowiedniej macierzy odbywa się poprzez szybkie obliczenie offsetu (imul r8, 24), co eliminuje konieczność stosowania wolnych instrukcji warunkowych (if/else) do wyboru typu daltonizmu.

3. Opis parametrów wejściowych dla programu:

1) Użytkownik wybiera z dysku zdjęcie w formacie JPG. Program wczytuje je i w pamięci zamienia na format mapy bitowej, aby móc edytować poszczególne piksele.

2) Liczba wątków (1 – 64): Użytkownik wpisuje lub wybiera suwakiem, na ile części ma zostać podzielony obraz.

3) Biblioteka (Silnik obliczeniowy): Przełącznik pozwalający wybrać, kto ma wykonać obliczenia:

- C++: Standardowa wersja programu.
- ASM: Szybka wersja napisana w asemblerze z użyciem instrukcji wektorowych.

4) Typ symulacji (Rodzaj daltonizmu): Wybór jednej z trzech wad wzroku, co zmienia wzory matematyczne używane przez program:

- Protanopia (niedowidzenie koloru zielonego).
- Deuteranopia (niedowidzenie koloru czerwonego).
- Tritanopia (niedowidzenie koloru niebieskiego).

4. Opis fragmentu asemblerowego kodu źródłowego biblioteki DLL z komentarzami:

Poniżej znajdują się najważniejsze fragmenty kodu asemblerowego kodu Główna pętla wektorowa i pętla skalarna z zaawansowanym adresowaniem:

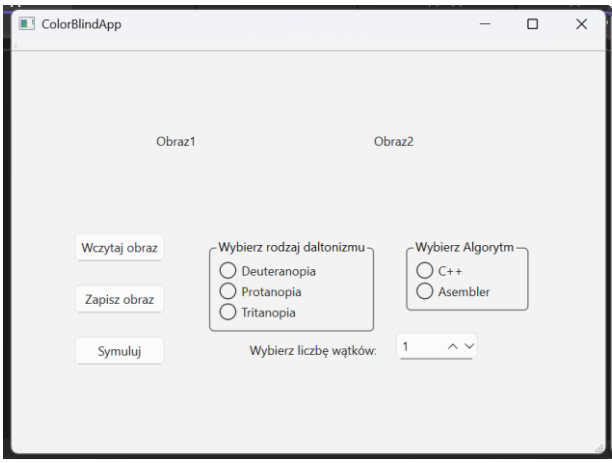
```
LoopVectors:
  ; ŁADOWANIE I ROZPAKOWANIE
  ; Pobranie 4 pikseli (16 bajtów) i rozszerzenie bajtów (0-255) do słów (16-bit).
  ; Konieczne do uniknięcia przepełnienia podczas mnożenia przez 1024.
  vpmovzxbw ymm0, XMMWORD PTR [rsi]
  vmovdqa ymm1, ymm0      ; Kopia danych dla kanału Green
  vmovdqa ymm2, ymm0      ; Kopia danych dla kanału Red
  ; OBLICZENIA MACIERZOWE
  ; Wzór: NewColor = (OldR*c1 + OldG*c2 + OldB*c3) / 1024
  ; Kanał Blue
  vpmaddwd ymm0, ymm0, ymm13 ; Mnożenie par liczb i sumowanie sąsiadów
  vphaddd  ymm0, ymm0, ymm0  ; Sumowanie poziome (zakończenie iloczynu skalarnego)
  vpsrad   ymm0, ymm0, 10    ; Dzielenie przez 1024 (Przesunięcie arytmetyczne w prawo z zachowaniem znaku)
  ; Kanał Green
  vpmaddwd ymm1, ymm1, ymm14
  vphaddd  ymm1, ymm1, ymm1
  vpsrad   ymm1, ymm1, 10
  ; Kanał Red
  vpmaddwd ymm2, ymm2, ymm15
  vphaddd  ymm2, ymm2, ymm2
  vpsrad   ymm2, ymm2, 10
  ; ZAPIS WYNIKÓW (Ekstrakcja i Saturacja)
  ; Procesor nie pozwala na bezpośredni zapis bajtów z YMM, więc wyciągane są do EAX.
  ; -- PIKSEL 1 (Indeks wektora: 0) --
  vpextrd eax, xmm0, 0     ; Pobranie wyniku Blue (32-bit int)
  call Clamp8              ; Saturacja do zakresu 0-255
  mov BYTE PTR [rsi], al   ; Zapis bajtu do pamięci
  vpextrd eax, xmm1, 0     ; Green
  call Clamp8
  mov BYTE PTR [rsi+1], al
  vpextrd eax, xmm2, 0     ; Red
  call Clamp8
  mov BYTE PTR [rsi+2], al
  ; -- PIKSEL 2 (Indeks wektora: 1) --
  vpextrd eax, xmm0, 1
```

```
    call Clamp8
    mov BYTE PTR [rsi+4], al
    vpextrd eax, xmm1, 1
    call Clamp8
    mov BYTE PTR [rsi+5], al
    vpextrd eax, xmm2, 1
    call Clamp8
    mov BYTE PTR [rsi+6], al
    ; Przełączenie na górną połówkę rejestrów YMM (dla pikseli 3 i 4)
    vextracti128 xmm3, ymm0, 1
    vextracti128 xmm4, ymm1, 1
    vextracti128 xmm5, ymm2, 1
    ; -- PIKSEL 3 --
    vpextrd eax, xmm3, 0
    call Clamp8
    mov BYTE PTR [rsi+8], al
    vpextrd eax, xmm4, 0
    call Clamp8
    mov BYTE PTR [rsi+9], al
    vpextrd eax, xmm5, 0
    call Clamp8
    mov BYTE PTR [rsi+10], al
    ; -- PIKSEL 4 --
    vpextrd eax, xmm3, 1
    call Clamp8
    mov BYTE PTR [rsi+12], al
    vpextrd eax, xmm4, 1
    call Clamp8
    mov BYTE PTR [rsi+13], al
    vpextrd eax, xmm5, 1
    call Clamp8
    mov BYTE PTR [rsi+14], al
    add rsi, 16            ; Przesunięcie wskaźnika o 4 piksele (16 bajtów)
    dec ecx                ; Zmniejszenie licznika pętli
    jnz LoopVectors        ; Skok na początek, jeśli licznik > 0
LoopScalar:
    ; -- DEMONSTRACJA ZAAWANSOWANEGO ADRESOWANIA --
    ; Adres = Base(RSI) + Index(R9) * Scale(4) + Displacement(0/1/2)
    ; Skala *4 wynika z faktu, że piksel ma 4 bajty.
    ; Pobranie piksela do rejestrów 32-bitowych (Zero Extend)
    movzx ebx, byte ptr [rsi + r9*4]    ; Old Blue
    movzx edx, byte ptr [rsi + r9*4 + 1] ; Old Green
    movzx edi, byte ptr [rsi + r9*4 + 2] ; Old Red
    ; Obliczenie Blue (Skalarnie)
    ; Wzór: (c0*B + c1*G + c2*R) >> 10
    movsx eax, word ptr [r11]    ; Pobranie c0 (z rozszerzeniem znaku)
    imul eax, ebx                ; Mnożenie przez Blue
    movsx r8d, word ptr [r11+2]  ; Pobranie c1
    imul r8d, edx                ; Mnożenie przez Green
    add eax, r8d                 ; Sumowanie
    movsx r8d, word ptr [r11+4]  ; Pobranie c2
    imul r8d, edi                ; Mnożenie przez Red
    add eax, r8d                 ; Sumowanie
    sar eax, 10                  ; Dzielenie przez 1024 (Arytmetyczne)
    call Clamp8                  ; Saturacja
    ; Zapis wyniku Blue (z użyciem skalowania)
    mov byte ptr [rsi + r9*4], al
    ; Obliczenie Green (Skalarnie)
    movsx eax, word ptr [r12]
    imul eax, ebx
    movsx r8d, word ptr [r12+2]
    imul r8d, edx
    add eax, r8d
    movsx r8d, word ptr [r12+4]
    imul r8d, edi
    add eax, r8d
```

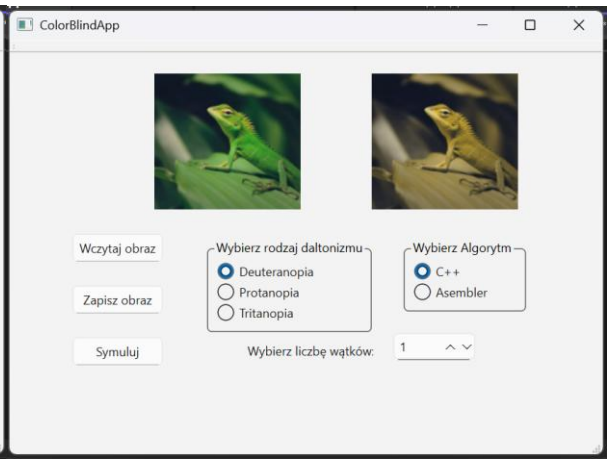
```
sar eax, 10
call Clamp8
; Zapis wyniku Green
mov byte ptr [rsi + r9*4 + 1], al
; Obliczenie Red (Skalarnie)
movsx eax, word ptr [r13]
imul eax, ebx
movsx r8d, word ptr [r13+2]
imul r8d, edx
add eax, r8d
movsx r8d, word ptr [r13+4]
imul r8d, edi
add eax, r8d
sar eax, 10
call Clamp8
; Zapis wyniku Red
mov byte ptr [rsi + r9*4 + 2], al
; -- Sterowanie pętlą skalarną --
inc r9d             ; Zwiększ indeks (kolejny piksel)
cmp r9d, r10d       ; Sprawdź czy osiągnęliśmy limit
jl LoopScalar       ; Jeśli indeks < limit, powtórz
```

5. Zrzuty ekranu wyglądu Interfejsu Użytkownika przed i w trakcie działania programu:

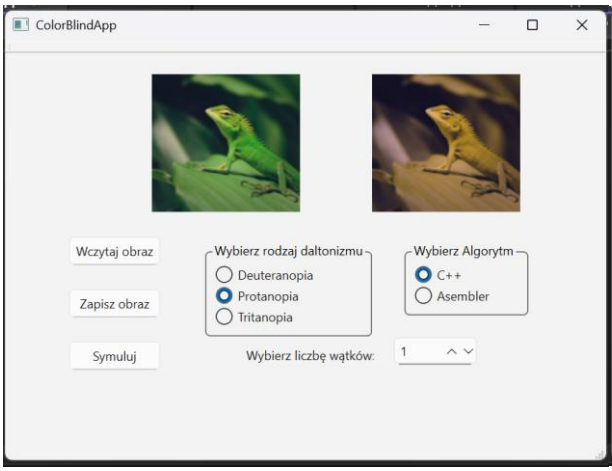
Widok przed wgraniem pliku:



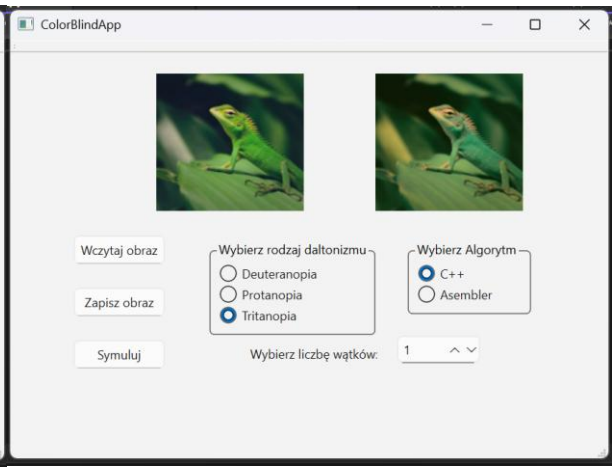
Symulacja Deuteranopii:



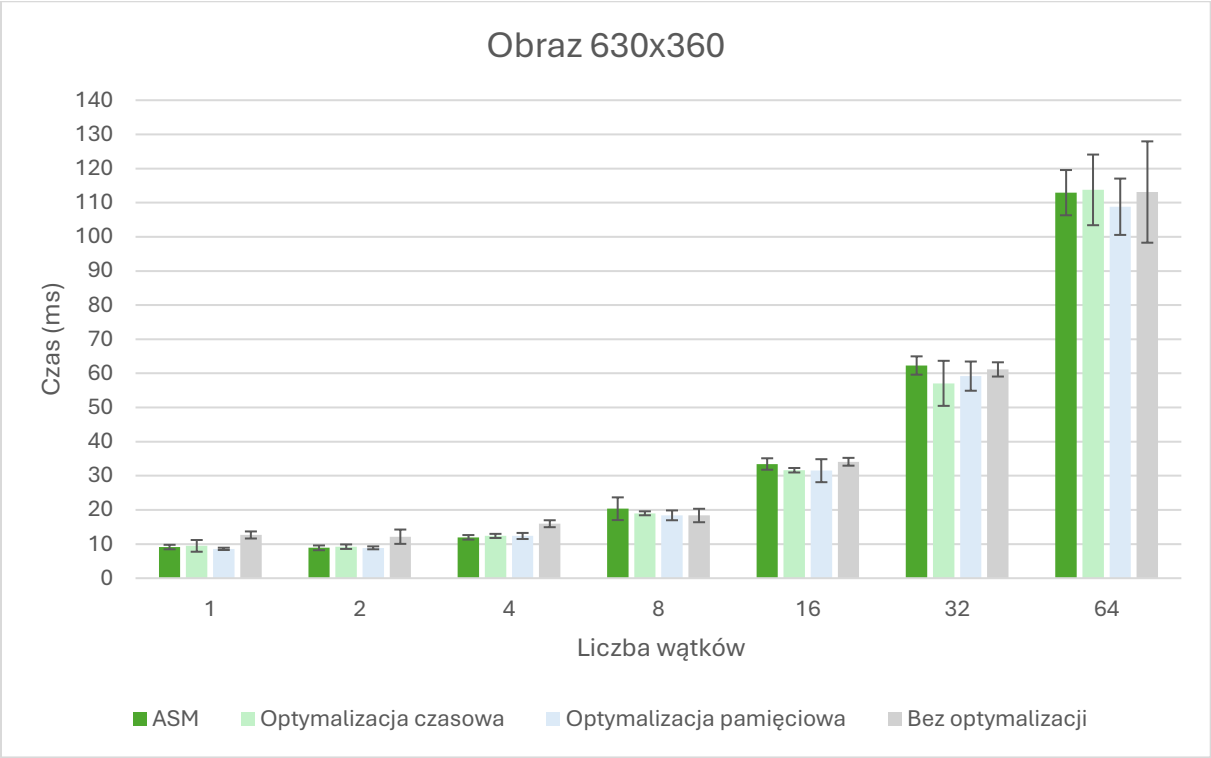
Symulacja Protanopii:

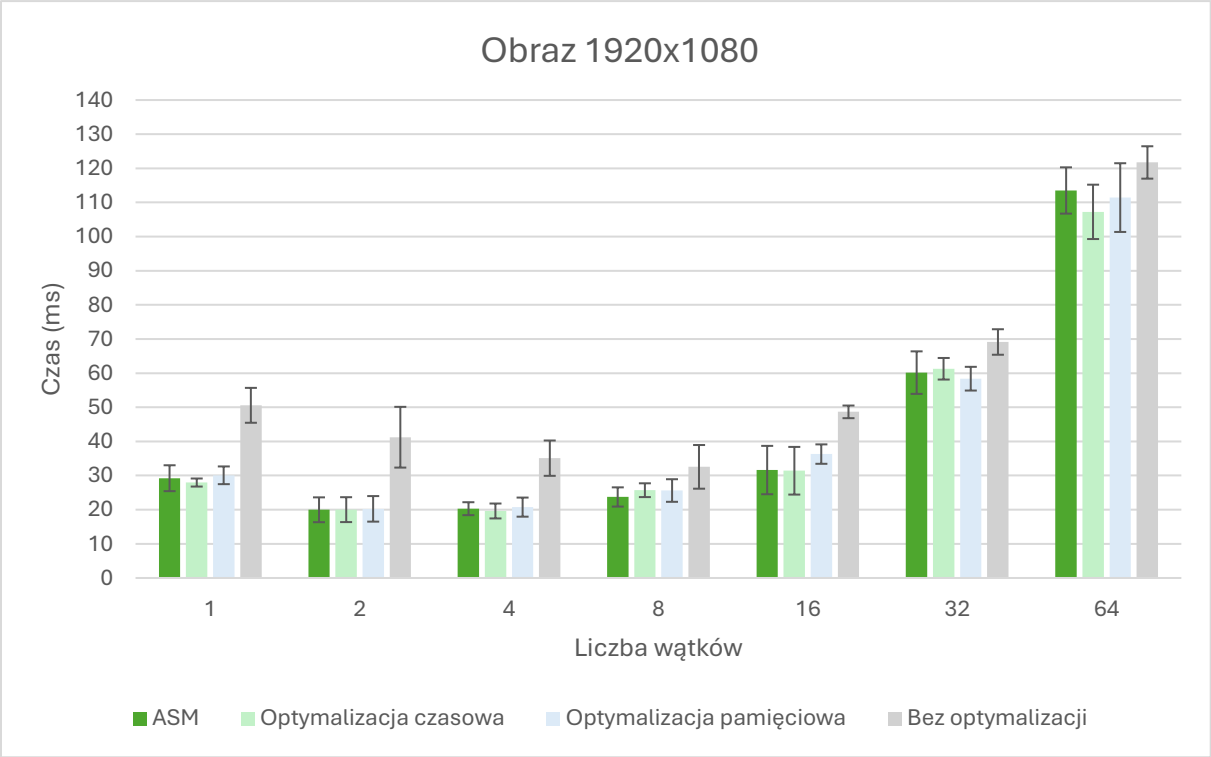


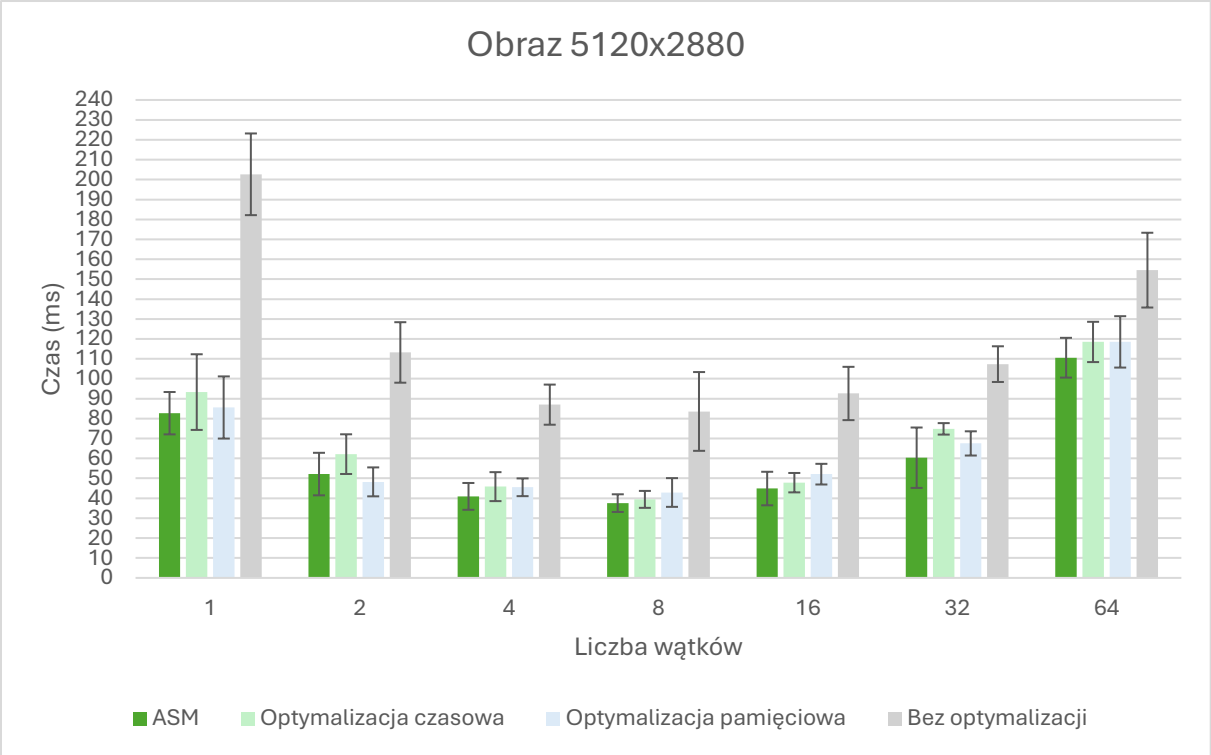
Symulacja Tritanopii:



6. Raport szybkości działania oparty na zgromadzonych danych pomiarowych w postaci wykresów graficznych wraz z odchyleniem standardowym w Excelu.







7. Opis testowania i uruchamiania programu (model procesora z opisem):

Testowanie przeprowadzono z użyciem procesora **Intel Core i7-1165G7**

Szczegółowe specyfikacje:

Liczba Rdzeni: 4

Liczba wątków: 8

Częstotliwość podstawowa: 2.8 GHz

Maksymalna częstotliwość: 4.7 GHz

Prędkość opony: 4 GT/s

Każdy z obrazów o rozmiarach(630x360, 1920x1080, 5120x2880) testowano dla dwóch implementacji ASM oraz C++. Każdy z poszczególnych obrazów był symulowany 6 razy dla danej liczby wątków w zakresie 1,2,4,8,16,32,64 a także sposobu implementacji oraz optymalizacji. Przykładowo obraz 1920x1080 był testowany 6 razy dla pojedynczej wartości wątków dla ASM co daje łącznie 42 pomiary dla samej implementacji w ASM. Ta czynność była powtarzana dla ASM, C++ z optymalizacją czasową, C++ z optymalizacją pamięciową oraz, C++ bez optymalizacji. Dla każdego z obrazów przeprowadzono 168 pomiarów co daje łącznie 504 pomiary dla 3 obrazów różnej wielkości. Na podstawie otrzymanych wyników sporządzono wykresy gdzie główne słupki to średnia 6 pomiarów czasu a słupki błędów to odchylenie standardowe tych pomiarów.

8. Wnioski z implementacji i działania programu z opisem interpretacji czasów wykonania w zależności od rozmiaru/ komplikacji danych wejściowych):

Realizacja projektu pozwoliła na praktyczne zweryfikowanie wydajności przetwarzania równoległego (wielowątkowość) oraz wektorowego (SIMD/ASM) w zadaniach graficznych.

Na podstawie zgromadzonych danych pomiarowych można sformułować następujące wnioski:

8.1. Wpływ rozmiaru danych na efektywność wielowątkowości

Zależność czasu wykonania od liczby wątków zmienia się diametralnie wraz ze wzrostem rozdzielczości obrazu:

1. Mały obraz (630x360):

   W tym scenariuszu wielowątkowość jest nieefektywna.

   Czas narzutu systemowego (utworzenie wątków, przydział pamięci dla każdego z nich, synchronizacja) jest dłuższy niż czas samych obliczeń.

   Najlepszy wynik uzyskano przy 1 lub 2 wątkach (ok. 8-10 ms). Użycie 64 wątków spowodowało ponad 10-krotny spadek wydajności (do ok.110 ms).

   Wniosek: Dla małych zbiorów danych algorytmy sekwencyjne są lepszym rozwiązaniem.

2. Średni obraz FHD(1920x1080):

   Ilość wątków w liczbie 2 dało najlepszą wydajność jednak 4 wątki miały bardzo zbliżony wynik

   Można dojść do wniosku że wraz ze wzrostem wielkości obrazu liczba wątków którą opłaca się użyć także wzrasta.

3. Duży obraz 5K(5120x2880):

   Tutaj widoczna jest klasyczna korzyść ze zrównoleglenia.

   Dla obrazu 5K (5120x2880) podział pracy na 8 wątków pozwolił skrócić czas przetwarzania ponad dwukrotnie względem jednego wątku (z ~82 ms do ~37 ms).

8.2. Porównanie Asembler (AVX2) vs C++:

Implementacja w Asemblerze wykazała nieznaczną przewagę nad kodem wysokiego poziomu, jednak jej skala zależy od konfiguracji kompilatora C++:

1. ASM vs C++ bez optymalizacji (/Od):

   Kod asemblerowy jest wielokrotnie szybszy (nawet 2-3 krotnie). Pokazuje to, jak nieefektywny jest domyślny kod generowany przez kompilator bez flag optymalizacyjnych.

2. ASM vs C++ z optymalizacją szybkościową (/O2):

   Wyniki wydajności ręcznie zoptymalizowanego kodu ASM są zbliżone do kodu C++ kompilowanego z flagą /O2. Jest to spowodowane autowektoryzacją (Auto-vectorization): Kompilator przy fladze /O2 analizuje pętle przetwarzające tablice i tam, gdzie to możliwe, automatycznie zastępuje instrukcje skalarne instrukcjami wektorowymi (SSE/AVX). Oznacza to, że kompilator "domyślił się", by zrobić to samo, co zostało napisane ręcznie w ASM.

3. ASM vs C++ z optymalizacją pamięciową (/O1):

Wyniki dla optymalizacji pamięciowej charakteryzują się zmiennością (czasami dorównują ASM, czasami są wolniejsze), co wynika z kompromisu inżynierskiego tej flagi kompilatora:

Zysk z Cache'u Instrukcji: Głównym celem tej optymalizacji jest redukcja rozmiaru kodu wynikowego. Mniejszy kod maszynowy efektywniej wykorzystuje pamięć podręczną instrukcji procesora (L1 Instruction Cache), co zmniejsza liczbę błędów chybienia (Cache Misses) i może przyspieszyć działanie programu w scenariuszach, gdzie kod jest często wykonywany (pętle).

8.3. Punkt nasycenia i "Ściana Pamięci" (Memory Wall)

Analiza wykresów dla dużych rozdzielczości (5K) wskazuje na występowanie wąskiego gardła w podsystemie pamięci RAM.

Mimo że procesor posiada moc obliczeniową, by przetwarzać dane szybciej (dzięki AVX2), przepustowość pamięci (Memory Bandwidth) ogranicza szybkość dostarczania nowych pikseli do rejestrów.

Dlatego różnice między ASM a C++ /O2 i C++ /O1 przy 8 wątkach dla dużego obrazu są mniejsze (rzędu kilku milisekund) niż przy 1 wątku.

8.4. Nadmierna wielowątkowość

Niezależnie od rozmiaru danych, uruchomienie programu na 64 wątkach (przy 8 wątkach logicznych procesora) zawsze prowadziło do drastycznej degradacji wydajności.

Jest to spowodowane kosztownym przełączaniem kontekstu (Context Switching) przez system operacyjny.

8.5. Wnioski końcowe:

Optymalna liczba wątków dla zadań obciążających CPU powinna być równa liczbie fizycznych/logicznych rdzeni procesora oraz jego liczbie wątków logicznych. Nowoczesne kompilatory przy włączonej optymalizacji stosują autowektoryzację używając instrukcji wektorowych przez co ich wydajność jest zbliżona się do ręcznie pisanego kodu asemblerowego.